HostingLint Security Model

For: NLnet NGI Zero Commons Fund Application

Date: February 2026

Core Principle: No Code Execution

HostingLint performs **static analysis only**. It never executes, evaluates, imports, or requires the analyzed code. All analysis happens on string content using regex pattern matching.

Security Guarantees

1. Read-Only File Access

HostingLint reads source files as plain text strings. It never writes to, modifies, or deletes analyzed files (except in explicit `--fix` mode where the user opts in to auto-fix).

```
Analyzed code → readFileSync() → string → regex matching → results
                                   ↑
                          No eval(), require(), or exec()
```

2. No Runtime Dependencies

HostingLint does **not** require:

- PHP interpreter (for analyzing PHP/WHMCS modules)
- Perl interpreter (for analyzing Perl/cPanel plugins)
- Docker runtime (for analyzing OpenPanel extensions)

This eliminates an entire class of supply chain and runtime vulnerabilities. A compromised PHP module cannot execute malicious code during analysis.

3. Pure Function Rules

Every rule is a pure function: `(context: RuleContext) => LintResult[]`

Rules are guaranteed to:

- Have no side effects (no file writes, no network calls, no process spawning)
- Take only a `RuleContext` input (code string, file path, config)
- Return only `LintResult[]` output
- Never throw exceptions (return empty array on error)

4. No Network Access

HostingLint makes zero network requests during analysis. There is no telemetry, no phone-home, no remote rule fetching. The tool operates entirely offline.

5. Minimal Dependencies

Package	Purpose	Scope
`commander`	CLI argument parsing	CLI only
`typescript`	Type system	Build time only
`vitest`	Testing	Dev only

No runtime dependencies in `@hostinglint/core`. The analysis engine is dependency-free.

Threat Model

Threats Mitigated

Threat	Mitigation
Malicious code execution during analysis	No eval/require/exec; string-only analysis
Supply chain attack via analyzed code	Analyzed code never enters Node.js module system
Information leakage	No network access, no telemetry
File system corruption	Read-only access (auto-fix is opt-in with preview)
Rule injection	Plugin rules validated against strict `Rule` interface
Denial of service via large files	Configurable file size limits, regex timeout

Threats Accepted

Threat	Status	Mitigation Plan
ReDoS (regex denial of service)	Accepted, low risk	Benchmark all regex patterns; add timeouts in M5
False negatives (missed vulnerabilities)	Inherent to regex approach	Expand rules, add AST option in M2
Plugin trust	User responsibility	Document plugin vetting process

Plugin Security

The plugin system (M2 milestone) will enforce:

1. **Interface validation**: Plugins must export `HostingLintPlugin` with `name: string` and `rules: Rule[]`
2. **Type checking**: Each rule is validated against the `Rule` interface before registration
3. **No dynamic evaluation**: Plugin rules are loaded as modules, not eval'd strings
4. **Duplicate detection**: Rules with conflicting IDs are rejected
5. **Documentation**: Plugin authors must document rule behavior and patterns

Auto-Fix Safety

The auto-fix feature (M2 milestone) will implement:

1. **Opt-in only**: `--fix` flag required, never automatic
2. **Dry-run default**: `--fix --dry-run` shows changes without applying
3. **Preview mode**: Interactive confirmation before each fix
4. **Backup**: Original files backed up before modification
5. **Range-based**: Fixes use precise line/column ranges, not file-wide replacements
6. **Validation**: Post-fix analysis confirms the fix resolved the issue

CI/CD Security

SARIF Output

HostingLint produces SARIF 2.1.0 output compatible with GitHub Code Scanning. SARIF files contain:

- Rule metadata (ID, description, severity)
- Finding locations (file, line, column)
- Fix suggestions (text only, not executable)

SARIF output contains **no source code** from analyzed files -- only file paths, line numbers, and rule-generated messages.

GitHub Actions Integration

The recommended GitHub Action workflow:

```
- name: Run HostingLint
  run: npx hostinglint check . --format sarif > results.sarif
- name: Upload SARIF
  uses: github/codeql-action/upload-sarif@v3
  with:
    sarif_file: results.sarif
```

No secrets, tokens, or credentials are required for analysis.

Comparison with Alternatives

Security Property	HostingLint	PHPStan	Perl::Critic
No runtime required	Yes	No (needs PHP)	No (needs Perl)
No code execution	Yes	Partial (loads code)	Partial (parses code)
No network access	Yes	Yes	Yes
Minimal dependencies	Yes (0 runtime)	No (50+)	No (20+)
Read-only analysis	Yes	Yes	Yes

Security Audit Plan (M5)

As part of the NLnet grant:

1. **Audit scope**: Core analysis engine, CLI, plugin loading, auto-fix engine
2. **Audit provider**: Radically Open Security (NLnet partner) or equivalent
3. **Focus areas**: regex safety, plugin isolation, auto-fix data integrity
4. **Timeline**: Month 9-10 of grant period
5. **Deliverables**: Audit report, remediation of all findings, public disclosure

---

Prepared for NLnet NGI Zero Commons Fund Application, February 2026